

Idea Discovery & Validation — Deep Dive

Real-Time Inventory Sync for Small Online + Offline Sellers

1. The Problem (stated precisely)

Small businesses that sell in more than one place — a physical shop plus an online store, or multiple online channels — keep separate, disconnected stock records. When a sale happens in one place, the other place doesn't know about it until someone manually updates it. This causes **overselling**: a customer buys something online that was actually already sold in the shop an hour earlier.

The scale of the problem, backed by real data:

- Inventory shrinkage and mismatch costs retailers an average of **1.44% of annual sales**, according to the National Retail Federation
- Stock mismatch between physical stores and online channels costs retailers up to **15% in lost sales** from overselling and stockouts
- Merchants running multi-channel operations can burn through **8-12 hours of manual reconciliation work every week** just trying to figure out which stock number is actually correct
- A real example from the research: a Shopify store shows 48 units, the warehouse reports 36, and accounting shows 42 — three different “truths,” and nobody knows which one is real

Why this happens (the root cause, in plain terms):

- The shop's cash register, the online store, and (sometimes) a warehouse spreadsheet are three separate systems that don't talk to each other
- Many small sellers update stock by hand, often once a day or at the end of a shift — during that gap, an item can sell out in-store but still show “available” online
- When a marketplace sale updates every 5-15 minutes instead of instantly, a burst of orders during a busy period can oversell multiple units before the system catches up

2. Why People Are Disturbed / What It Actually Costs Them

This isn't just an annoyance — it's a chain reaction of real damage:

1. **A customer buys an out-of-stock item online** → the seller has to cancel the order and issue a refund
2. **The customer loses trust** and often doesn't come back, so it's not just one lost sale — it's a lost future customer
3. **The seller loses confidence in their own numbers** — when you can't trust your stock count, every decision (what to restock, what to promote, what to discount) becomes a guess instead of a fact

4. **Hidden costs pile up quietly:** expedited shipping to fix a mistake, refunds, replacement items, and “dead stock” (things sitting unsold because nobody trusted the numbers enough to promote them)
5. **On marketplaces like Amazon or Etsy,** repeated cancellations from overselling can even risk account suspension — so this genuinely threatens a seller’s ability to keep operating there at all

3. Who Has This Problem (Target Audience)

Primary persona: “The small multi-channel seller”

- Runs a small shop *and* sells online (own website, Instagram/Facebook shop, or a marketplace)
- Has under 50-200 products (SKUs) — too many to track by memory, too few to justify enterprise software
- Currently tracks stock through a mix of a physical register, a notebook, or a basic spreadsheet
- Does not have a dedicated inventory staff member — the owner or one employee does it manually between customers

Secondary persona: “The growing online-only seller”

- Sells across 2-3 online channels (own store + a marketplace) without a physical shop
- Experiences the same mismatch problem purely between different online platforms

Why this audience is reachable: small retail shops and online sellers are highly visible and locatable — through local market associations, Instagram business pages, and small-business Facebook groups — not scattered anonymously across the internet.

4. Why This Is Still Unsolved (the real gap)

This is the most important part: **solutions already exist, but they fail small sellers specifically.**

Existing tool	Why it fails small sellers
Cin7 Core / NetSuite / Linnworks	Built for large, high-volume businesses — starts at \$349/month , steep learning curve, “overkill for small maker businesses”
Skubana / Webgility	Advanced forecasting and accounting features small sellers don’t need, priced accordingly
Tools with hidden/quote-based pricing	Research explicitly warns: “tools that require you to contact sales for a quote are almost always more expensive”
Per-user pricing tools	A tool costing “\$29/user/month sounds cheap” until a small team of 5 pays \$1,740/year just in seat fees, before any real features

Manual spreadsheets / registers

Free, but this *is* the actual root cause of the mismatch problem in the first place — no automation, relies on someone remembering to update it

The real gap: almost every real inventory-sync tool is built and priced for businesses already doing high order volume. A small shop with 50-150 products and a handful of daily sales is treated as an afterthought — either priced out entirely, or sold a tool so complex it doesn't get properly used, so the mismatch problem continues anyway.

Why it's newly solvable now: AI-assisted development (tools like Cursor) make it realistic for a solo developer to build a simple, focused sync tool quickly — something that used to require a funded team is now buildable as a student project.

5. Proposed Solution

A simple, affordable inventory sync tool built for **one specific type of seller**: someone with a physical shop and one online channel (or two small online channels), under roughly 150 products.

Core idea: one shared, single “source of truth” stock count that every sale — whether from the shop counter or an online order — subtracts from immediately. Not two separate systems trying to stay in sync; one number, always accurate, viewed from two places.

6. Core MVP Features (3-5, realistic scope)

1. **Single shared stock list:** one master inventory list with product name, quantity, and price
2. **Manual counter-sale entry:** a fast, simple way for the shop owner to log an in-store sale in seconds (tap the product, tap “sold”) — this is the step that's usually skipped or delayed in real life, so it has to be genuinely fast
3. **Automatic online-order deduction:** when an online order comes in, stock is deducted automatically — no manual double-entry
4. **Low-stock alert:** a simple notification when an item drops below a set quantity, so the seller can reorder before running out completely
5. *(Optional v2)* Basic sales history/report — what sold, when, from which channel

7. User Journey (step-by-step)

1. Seller sets up their product list once (name, starting quantity, price) — kept intentionally quick, under 15 minutes for ~50 products
2. A customer buys an item in the physical shop → seller taps the product in the app, marks it sold → stock count updates instantly
3. At the same time, a customer browsing the online store sees the *correct*, updated stock — no risk of buying something that's actually gone
4. An online order comes in → stock automatically deducts, no manual step needed
5. When an item gets low, the seller gets a simple alert to reorder

8. Tech Stack (realistic for a student to build)

- **Frontend:** React + TypeScript + Vite (same stack as my portfolio website)
- **Backend/database:** Supabase or Firebase — handles real-time updates well, which is the core requirement here (when one person changes stock, everyone viewing it sees the update instantly)
- **Online store connection (v1):** start with a simple built-in online storefront view rather than integrating with Shopify/WooCommerce APIs immediately — this avoids a lot of early complexity while still proving the core “one shared number” concept
- **Real-time sync mechanism:** Supabase’s real-time subscriptions (or Firebase’s real-time database) — this is the actual technical heart of the product

9. Data & Privacy Considerations

- Only business-owned data (product names, quantities, prices) — no customer personal data required for the MVP
- If a contact form or online storefront is added later, standard basic data handling applies (name, email/phone for order contact only)

10. Risks & Honest Challenges

- **Adoption habit risk:** the whole system only works if the shop owner actually logs in-store sales *immediately* — if they forget or delay, the same old mismatch problem comes right back. The tool’s design has to make logging a sale faster than *not* logging it.
- **Internet dependency:** if the shop’s internet goes down, real-time sync can’t happen — research shows the strongest tools handle this with “offline-first” systems that record locally and sync once connection returns; this is a good v2 goal, not required for MVP
- **Trust in a new/unknown tool:** small sellers who’ve been burned by clunky software before may be hesitant — starting with one real shop as a pilot (exactly like your handbook’s validation approach) builds trust through a real result, not just a pitch

11. How to Validate Before Building

1. Talk to 5-10 small shop owners who also sell online (or on Instagram/Facebook) — ask specifically: “Has a customer ever ordered something online that turned out to be already sold in the shop?” This single question tests if the problem is real for them.
2. Manually reconcile one real shop’s online vs. in-store stock by hand for a week, and time how long it takes — this proves the size of the pain before writing any code
3. If a shop owner says “yes, this happens to me” and “I’d want something like this,” that’s your green light to build

12. Success Metrics for a v1

- Zero (or near-zero) overselling incidents for the pilot shop during the testing period
- Time saved on manual stock reconciliation, measured against the

“by hand” baseline from validation

- The shop owner actually logging in-store sales consistently (this is the real proof the tool works in daily life, not just in theory)
-

Why This Is a Strong Idea (summary)

- **The problem is real and measurably expensive** (1.44%–15% of sales lost, 8-12 hours/week wasted) — not a guess
- **The existing market has a genuine, provable gap:** solutions exist, but they’re priced and built for large businesses, leaving small sellers with either nothing or something they can’t afford or won’t properly use
- **It’s different from a generic support chatbot** (like your classmate’s project) — this solves a completely separate, concrete money-loss problem, so there’s no overlap
- **It’s realistically buildable** by a student using tools you already know (React, Vite, and a real-time backend like Supabase)
- **It’s easy to validate cheaply** — a handful of conversations with real shop owners before writing any code tells you immediately if this is worth building